

TRƯỜNG ĐẠI HỌC VINH
VIỆN KỸ THUẬT VÀ CÔNG NGHỆ



ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT NÂNG CAO

NGHIÊN CỨU VÀ ỨNG DỤNG THUẬT TOÁN
ANT COLONY SYSTEM TRONG
BÀI TOÁN ĐỊNH TUYẾN XE ĐỘNG

GVHD: TS. Nguyễn Thị Uyên

Học phần: Cấu trúc dữ liệu và giải thuật nâng
cao_LT_03_(TC_CNTT)

Nhóm SVTH: 1. Nguyễn Trọng Đức (NT) – 225748020110305

2. Nguyễn Đình Quý – 225748020110117

Nghệ An, 3/2026

LỜI CẢM ƠN

Đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sự tri ân sâu sắc đến các thầy cô Trường Đại học Vinh, đặc biệt là các thầy cô thuộc Viện Kỹ thuật và Công nghệ đã tạo điều kiện thuận lợi để chúng em thực hiện và hoàn thành đề tài nghiên cứu chuyên ngành.

Trong suốt quá trình thực hiện đề tài ***“Nghiên cứu và ứng dụng thuật toán Ant Colony System trong bài toán định tuyến xe động”***, chúng em đã nhận được sự hướng dẫn, hỗ trợ và chỉ bảo tận tình từ giảng viên hướng dẫn TS.Nguyễn Thị Uyên. Cô đã giúp chúng em định hướng nội dung nghiên cứu, củng cố kiến thức chuyên môn và đưa ra nhiều góp ý quý báu để chúng em hoàn thiện hơn sản phẩm và báo cáo của mình. Chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy/cô vì những sự giúp đỡ nhiệt tình trong suốt thời gian qua.

Dù đã cố gắng hoàn thiện tốt nhất trong khả năng của mình, nhưng do kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót trong quá trình thực hiện. Chúng em rất mong nhận được những ý kiến đóng góp từ quý thầy cô và các bạn để đề tài được hoàn thiện tốt hơn.

Chúng em xin chân thành cảm ơn !

Nhóm thực hiện đề tài

Nhóm 3

MỤC LỤC

LỜI CẢM ƠN	1
DANH MỤC CÁC HÌNH	4
LỜI MỞ ĐẦU	5
CHƯƠNG 1: MÔ TẢ BÀI TOÁN	7
1.1. Giới thiệu về bài toán định tuyến xe động (Dynamic Vehicle Routing Problem – DVRP).....	7
1.2. Yêu cầu của bài toán định tuyến xe	9
1.3. Yêu cầu của bài toán định tuyến xe động.....	12
1.4. Ứng dụng của bài toán định tuyến xe động.....	15
1.5. Các nghiên cứu liên quan đến bài toán định tuyến xe động	16
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN.....	20
2.1. Khái quát về thuật toán Ant Colony System (ACS)	20
2.2. Mô hình hóa của bài toán định tuyến xe động.....	21
2.3. Phân tích và thiết kế bộ quản lý sự kiện (Events manager).....	22
2.4. Phân tích và thiết kế thuật toán Ant Colony System	26
2.5. Phân tích và thiết kế thủ tục bảo toàn pheromone (Pheromone conservation)	32
CHƯƠNG 3: CÀI ĐẶT VÀ THỰC NGHIỆM ĐÁNH GIÁ	33
3.1. Thiết lập bộ dữ liệu của bài toán định tuyến xe động.....	33
3.2. Thiết lập tham số của thuật toán ACS	34
3.3. Đánh giá thuật toán ACS với các thuật toán khác	36
3.4. Môi trường phát triển.....	38
3.5. Xây dựng và cài đặt hệ thống	39

<i>3.6. Kiểm thử và đánh giá hệ thống</i>	42
KẾT LUẬN	47
TÀI LIỆU THAM KHẢO	48

DANH MỤC CÁC HÌNH

Hình 1.1. Ví dụ về bài toán định tuyến xe.....	12
Hình 1.2. Ví dụ về bài toán định tuyến xe động.....	13
Hình 2.1. Mô hình hóa của bài toán định tuyến xe động.....	19
Hình 3.1. Thí nghiệm đánh giá tham số n_{ts}	35
Hình 3.2. Thí nghiệm đánh giá tham số γ_r	35
Hình 3.3. Thí nghiệm so sánh thuật toán.....	37
Hình 3.4. Biểu đồ biểu diễn kết quả kiểm thử với số lượng nhỏ khách hàng.....	44
Hình 3.5. Biểu đồ biểu diễn kết quả kiểm thử với thuật toán GRASP.....	45

LỜI MỞ ĐẦU

Trong bối cảnh nền kinh tế số và thương mại điện tử phát triển mạnh mẽ, nhu cầu vận chuyển và giao nhận hàng hóa ngày càng gia tăng cả về quy mô lẫn tốc độ. Các doanh nghiệp logistics không chỉ phải đảm bảo giao hàng đúng thời gian mà còn phải tối ưu chi phí vận hành, đặc biệt là chi phí nhiên liệu và thời gian di chuyển. Một trong những bài toán cốt lõi trong lĩnh vực tối ưu vận tải là bài toán định tuyến xe (Vehicle Routing Problem – VRP), trong đó mục tiêu là xây dựng các tuyến đường hợp lý cho đội xe nhằm phục vụ một tập khách hàng với chi phí thấp nhất.

Tuy nhiên, trong thực tế, môi trường vận hành hiếm khi mang tính “tĩnh”. Các đơn hàng có thể phát sinh liên tục trong ngày, thông tin giao thông thay đổi theo thời gian thực, và doanh nghiệp phải điều chỉnh kế hoạch vận chuyển một cách linh hoạt. Điều này dẫn đến sự ra đời của bài toán định tuyến xe động (Dynamic Vehicle Routing Problem – DVRP). Khác với VRP truyền thống, DVRP yêu cầu hệ thống phải liên tục cập nhật và tái tối ưu khi có đơn hàng mới xuất hiện. Đây là một bài toán phức tạp thuộc lớp NP-hard, đòi hỏi các phương pháp giải hiệu quả thay vì tìm lời giải chính xác tuyệt đối.

Trong những năm gần đây, các phương pháp tối ưu hóa metaheuristic đã chứng minh được hiệu quả trong việc giải quyết các bài toán tối ưu tổ hợp quy mô lớn. Một trong những phương pháp nổi bật là thuật toán Tối ưu hóa đàn kiến (Ant Colony Optimization – ACO), lấy cảm hứng từ hành vi tìm đường của loài kiến trong tự nhiên. Thuật toán Ant Colony System (ACS), một biến thể cải tiến của ACO, đã cho thấy khả năng tìm kiếm lời giải chất lượng cao trong nhiều bài toán định tuyến và tối ưu hóa tổ hợp khác nhau nhờ cơ chế cập nhật và bảo tồn pheromone thông minh.

Xuất phát từ tầm quan trọng thực tiễn của bài toán định tuyến xe động và tiềm năng của các phương pháp tối ưu hóa đàn kiến, dựa theo dữ liệu tài liệu nghiên cứu “*Ant Colony System for a Dynamic Vehicle Routing Problem*” của các

tác giả **R.Montemanni, L.M.Gambardella, A.E.Rizzoli** và **A.V.Donati**, báo cáo này tập trung nghiên cứu mô hình bài toán DVRP và phân tích cơ chế hoạt động của thuật toán Ant Colony System trong việc giải quyết bài toán này. Đồng thời, báo cáo cũng trình bày cách áp dụng ACS vào môi trường định tuyến xe động thông qua cơ chế chia lát thời gian và bảo tồn thông tin tối ưu giữa các giai đoạn tính toán.

Vì vậy, nội dung báo cáo được xây dựng với mục tiêu ***nghiên cứu và ứng dụng thuật toán Ant Colony System cho bài toán định tuyến xe động***, từ đó đánh giá hiệu quả và khả năng áp dụng của phương pháp trong các hệ thống giao hàng thực tế.

CHƯƠNG 1: MÔ TẢ BÀI TOÁN

1.1. Giới thiệu về bài toán định tuyến xe động (Dynamic Vehicle Routing Problem – DVRP)

1.1.1. Giới thiệu khái quát về bài toán định tuyến xe (Vehicle Routing Problem – VRP)

Với sự phát triển của xã hội hiện nay, ngành logistics và quản lý vận tải đóng một vai trò vô cùng quan trọng trong việc liên thông hàng hóa giữa các khu vực. Trong đó, việc lập kế hoạch di chuyển cho các phương tiện giao hàng là một bài toán nan giải và có tính cấp bách. Bởi sự xuất hiện hàng loạt của những doanh nghiệp và nhu cầu của người tiêu dùng hiện nay đang tạo ra những vấn đề trong quá trình vận chuyển, chẳng hạn như việc các doanh nghiệp phục vụ một lượng khách hàng ở nhiều vị trí khác nhau trong khi nguồn lực (số xe, thời gian, chi phí vận hành) lại có hạn. Vì vậy, mục tiêu trong việc giải quyết bài toán định tuyến xe chính là thiết lập và xây dựng một lộ trình hợp lý sao cho *“một đội xe có giới hạn về tải trọng để phục vụ một tập khách hàng sao cho chi phí là nhỏ nhất”* [1] trở thành một bài toán tối ưu quan trọng, hay nói một cách khác, chính là ***bài toán về định tuyến xe – Vehicle Routing System***.

Bài toán định tuyến xe (Vehicle Routing System – VRP) là một trong những tổ hợp tối ưu hóa khó khăn trong vận hành và quản lý vận tải, phân phối và logistics. Bài toán này lần đầu xuất hiện trong một bài báo nghiên cứu của hai tác giả là George Dantzig và John Ramser vào năm 1959 về vấn đề khả năng điều phối xe cho mục đích chở xăng dầu. Nói đơn giản, bài toán VRP tìm cách trả lời cho câu hỏi: “Làm thế nào để các xe xuất phát từ kho hàng đi phục vụ tất cả khách hàng với chi phí thấp nhất?” (*“What is the optimal set of routes for a fleet of vehicles to traverse in order to deliver to a given set of customers?”*). “Chi phí” ở đây có thể là tổng quãng đường di chuyển, tổng thời gian vận chuyển, chi phí nhiên liệu hoặc sự kết hợp của nhiều yếu tố khác.

Một bài toán VRP được định nghĩa dưới dạng mô hình cơ bản với những thành phần như sau:

- **Kho hàng (Depot):** đây là nơi hàng hóa được lưu trữ và cũng là nơi các xe bắt đầu hành trình.
- **Tập khách hàng:** bao gồm lượng khách hàng có vị trí địa lý cụ thể và có nhu cầu phục vụ cụ thể.
- **Đội xe vận chuyển:** bao gồm nhiều xe có sức chứa và lộ trình khác nhau.
- **Tuyến đường di chuyển:** giữa kho và khách hàng tồn tại khoảng cách và thời gian di chuyển.

Trong thực tế, bài toán VRP đã xuất hiện trong rất nhiều hệ thống vận hành hàng ngày, đặc biệt là trong các doanh nghiệp có hoạt động giao nhận hàng hóa. Một số sàn thương mại điện tử lớn như Shopee, GHN, ViettelPost,... mỗi ngày phải phục vụ cho hàng trăm đến hàng nghìn khách hàng, cho nên họ cần phải tối ưu lộ trình để có thể tiết kiệm chi phí và thời gian. Các hệ thống giao đồ ăn nhanh như GrabFood, Baemin,... mỗi ngày có một lượng lớn đơn hàng và tài xế bất kỳ, cho nên họ cần phải sắp xếp và bàn giao đơn hàng thật hợp lý cho các tài xế để có thể giảm thời gian giao hàng và tối ưu chi phí cho người giao hàng. Không những thế, bài toán VRP còn xuất hiện trong lĩnh vực vận chuyển hàng công nghiệp như phân phối gas, phân phối nhiên liệu, vận tải hàng hóa,... giúp tối ưu thời gian giao, lộ trình vận chuyển và chi phí vận hành cho các doanh nghiệp, tổ chức.

Bài toán định tuyến xe tĩnh (Static Vehicle Routing Problem – Static VRP) là một trong những “biến thể” xuất phát từ bài toán VRP. Điểm đặc trưng của bài toán này là toàn bộ thông tin về khách hàng/đơn hàng đều được biết trước khi bắt đầu lập kế hoạch (a priori), do đó việc tối ưu hóa có thể được thực hiện một lần duy nhất trước khi xe khởi hành. Trong phạm vi nghiên cứu của báo cáo này, ta có thể coi thuật ngữ bài toán VRP dùng để chỉ bài toán định tuyến xe tĩnh, trong đó đều có một mục tiêu chính là tối thiểu hóa tổng chi phí vận chuyển.

1.1.2. Giới thiệu khái quát về bài toán định tuyến xe động (Dynamic Vehicle Routing Problem – DVRP)

Với những thách thức xuất hiện trong quá trình nghiên cứu và phát triển bài toán VRP, những “biến thể” khác nhau về bài toán VRP ra đời. Trong đó, **bài toán định tuyến xe động (Dynamic Vehicle Routing Problem)**, hay còn được gọi là **bài toán định tuyến trực tuyến (On-line VRP)**, xuất hiện trong tình hình nhu cầu của người tiêu dùng càng tăng và sự có mặt của những sàn thương mại điện tử, trong những thách thức như yêu cầu thay đổi của khách hàng, nhu cầu cập nhật liên tục,..., cho phép thu thập và xử lý thông tin theo thời gian thực. Bài toán VRP động được xây dựng nhằm mô tả quá trình tổ chức và điều phối phương tiện vận chuyển trong môi trường mà thông tin thay đổi theo thời gian. Trong mục này, ta sẽ tìm hiểu và phân tích những điểm đặc trưng của bài toán, từ đó hiểu được vấn đề cốt lõi và triển khai thuật toán giải quyết vấn đề.

1.2. Yêu cầu của bài toán định tuyến xe

1.2.1. Mô hình hóa bài toán định tuyến xe

Xét trong phạm vi nghiên cứu xoay quanh báo cáo “*Ant Colony System for a Dynamic Vehicle Routing Problem*”, bài toán VRP được mô tả giả định như sau:

- n khách hàng cần được phục vụ từ một kho duy nhất (depot), mỗi khách hàng i yêu cầu một lượng hàng hóa q_i .
- một đội xe gồm v phương tiện, mỗi phương tiện a có tải trọng tối đa Q_a dùng để vận chuyển hàng hóa.
- mỗi khách hàng có thời gian cần thiết để phục vụ s_i .

Cho nên, với mỗi lời giải cho bài toán VRP đều được xem là **một tập hợp các tuyến đường (tours)**. Về mặt toán học, bài toán VRP được mô hình hóa thông qua **một đồ thị có hướng đầy đủ có trọng số** như sau:

$$G = (V, A)$$

trong đó:

- $V = \{0, 1, \dots, n\}$ là tập các đỉnh. Đỉnh 0 đại diện cho kho hàng (depot). Các đỉnh $(1, \dots, n)$ đại diện cho khách hàng.
- $A = \{(i, j) \mid i, j \in V\}$ là tập các cung (đường đi giữa các điểm), mỗi cung được gán một thời gian di chuyển tối thiểu là tt_{ij} .

1.2.2. Thách thức của bài toán định tuyến xe

Xét theo bài toán giả định trên, với mục tiêu là tìm một tập hợp các tuyến đường khả thi sao cho **tổng thời gian di chuyển là nhỏ nhất**, với điều kiện là thỏa mãn các ràng buộc sau:

- mỗi đỉnh (khách hàng) được **ghé thăm chính xác một lần** (tức là chỉ nằm trong một tuyến duy nhất)
- mỗi tuyến a bắt đầu và kết thúc tại kho (đỉnh 0)
- tổng nhu cầu hàng hóa của các khách hàng trong mỗi tuyến không vượt quá tải trọng của phương tiện tương ứng Q_a .

Cho nên, trong suốt quá trình nghiên cứu và phát triển, các tác giả đã rút ra những thách thức liên quan đến bài toán VRP như sau:

- Số lượng phương án lộ trình rất lớn: số lượng khách hàng càng tăng, số cách sắp xếp tuyến đường rất nhanh, việc tìm lời giải tối ưu trong thời gian ngắn sẽ trở nên khó khăn.

- Nhiều ràng buộc cần phải thỏa mãn: bài toán VRP cần phải đảm bảo các điều kiện bên lề như sức chứa của xe, thời gian giao hàng,...

- Dữ liệu và môi trường thay đổi liên tục: trong quá trình vận chuyển hàng hóa có thể phát sinh đơn hàng mới, khách hàng thay đổi địa điểm, tình trạng giao thông thay đổi,...

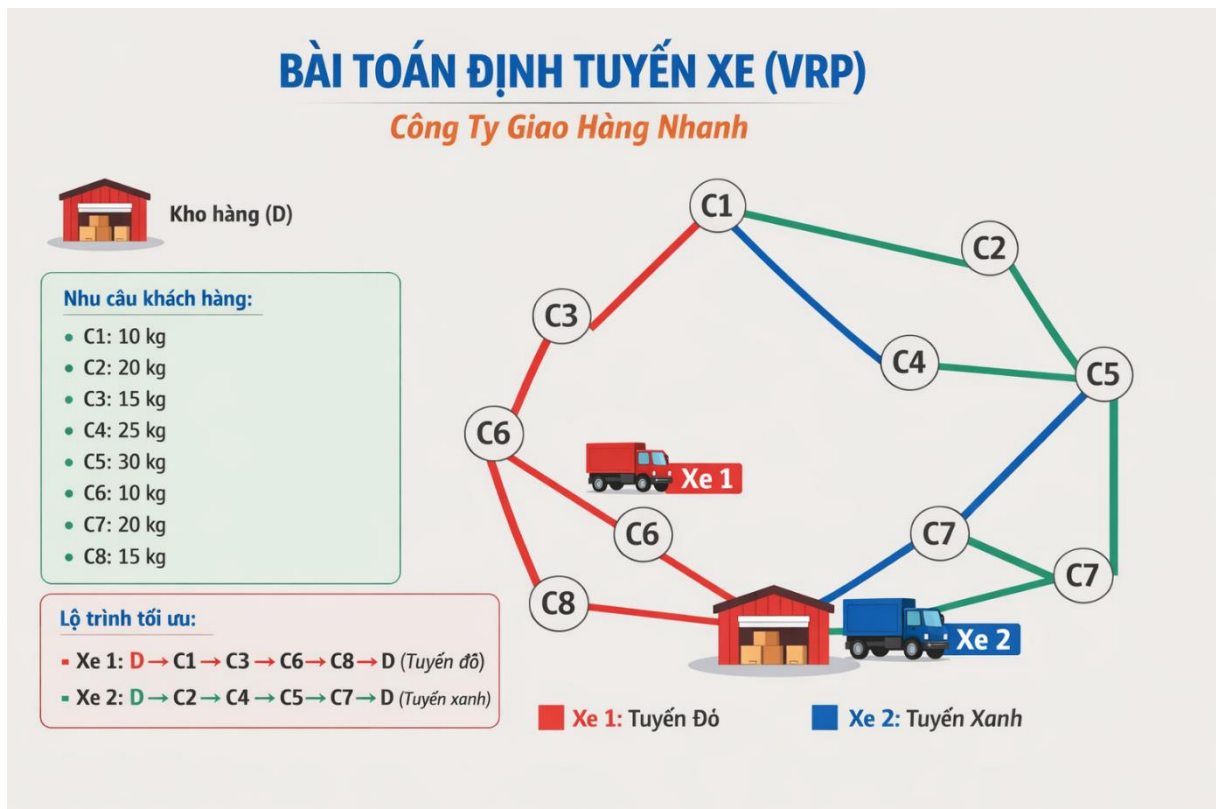
- Yêu cầu tối ưu nhiều mục tiêu cùng lúc: các doanh nghiệp, tổ chức thường muốn tối ưu nhiều yếu tố và cùng một lúc khiến bài toán trở nên khó khăn.

1.2.3. Ví dụ về bài toán định tuyến xe

Giả sử có một công ty giao hàng trong thành phố, vào mỗi buổi sáng đều bắt đầu xuất phát tại một vị trí gọi là kho hàng D. Công ty này có 2 xe giao hàng, với sức chứa tối đa 60kg, trong ngày có 8-10 khách hàng cần nhận hàng. Mỗi khách hàng có vị trí khác nhau trong thành phố và khối lượng hàng cần giao khác nhau. Ví dụ dữ liệu về một ngày giao hàng của công ty đó như sau:

Khách hàng	Lượng hàng cần giao
C1	10kg
C2	20kg
C3	15kg
C4	25kg
C5	30kg
C6	10kg
C7	20kg
C8	15kg

Dựa vào những dữ liệu trên, ta xác định những thành phần cơ bản của bài toán VRP: vị trí D chính là kho hàng (depot), là điểm xuất phát của tất cả xe, với tập khách hàng từ C1 đến C8, mỗi khách hàng có vị trí và nhu cầu khác nhau, trong khi đó đội xe bao gồm 2 xe, với ràng buộc là sức chứa tối đa là 60kg. Vấn đề đặt ra ở đây là: nếu không tối ưu lộ trình, sẽ gây nên tình trạng thiếu nhiên liệu, tăng thời gian giao hàng và làm giảm hiệu suất vận hành. Cho nên, ta cần phải áp dụng bài toán VRP để có thể giải bài toán này. Dưới đây là một trong những lời giải tối ưu để giải quyết vấn đề trên:



Hình 1.1. Ví dụ về bài toán định tuyến xe

1.3. Yêu cầu của bài toán định tuyến xe động

1.3.1. Bối cảnh của bài toán định tuyến xe động

Xét theo phạm vi nghiên cứu của báo cáo “*Ant Colony System for a Dynamic Vehicle Routing Problem*”, giả định một case study như sau: một hệ thống liên lạc giữa **bộ phận điều phối (dispatcher)**, chẳng hạn như trụ sở công ty và **các tài xế**. Bộ phận điều phối có trách nhiệm định kỳ thông báo cho tài xế về các điểm giao hàng mới được phân công. Từ đó, trong suốt ngày làm việc, mỗi tài xế luôn biết được những khách hàng tiếp theo mình phải phục vụ. Tức là, trong quá trình vận chuyển, họ sẽ bắt đầu và kết thúc sau khi đã giao hết những đơn hàng phát sinh tại bộ phận điều phối, đồng thời thỏa mãn các ràng buộc của một bài toán VRP.

Cho nên, xét theo độ phù hợp với thuật toán mà chúng tôi đang nghiên cứu và tìm hiểu, có thể xác định những bài toán thực tế chính có đặc điểm trên:

Hệ thống trung chuyển (feeder systems): vận chuyển theo yêu cầu trong phạm vi địa phương, phục vụ khách hàng đến một hệ thống giao thông lớn hơn tại một điểm trung chuyển nhất định [2], và rời khỏi kho khi không có hàng của những đơn hàng phát sinh.

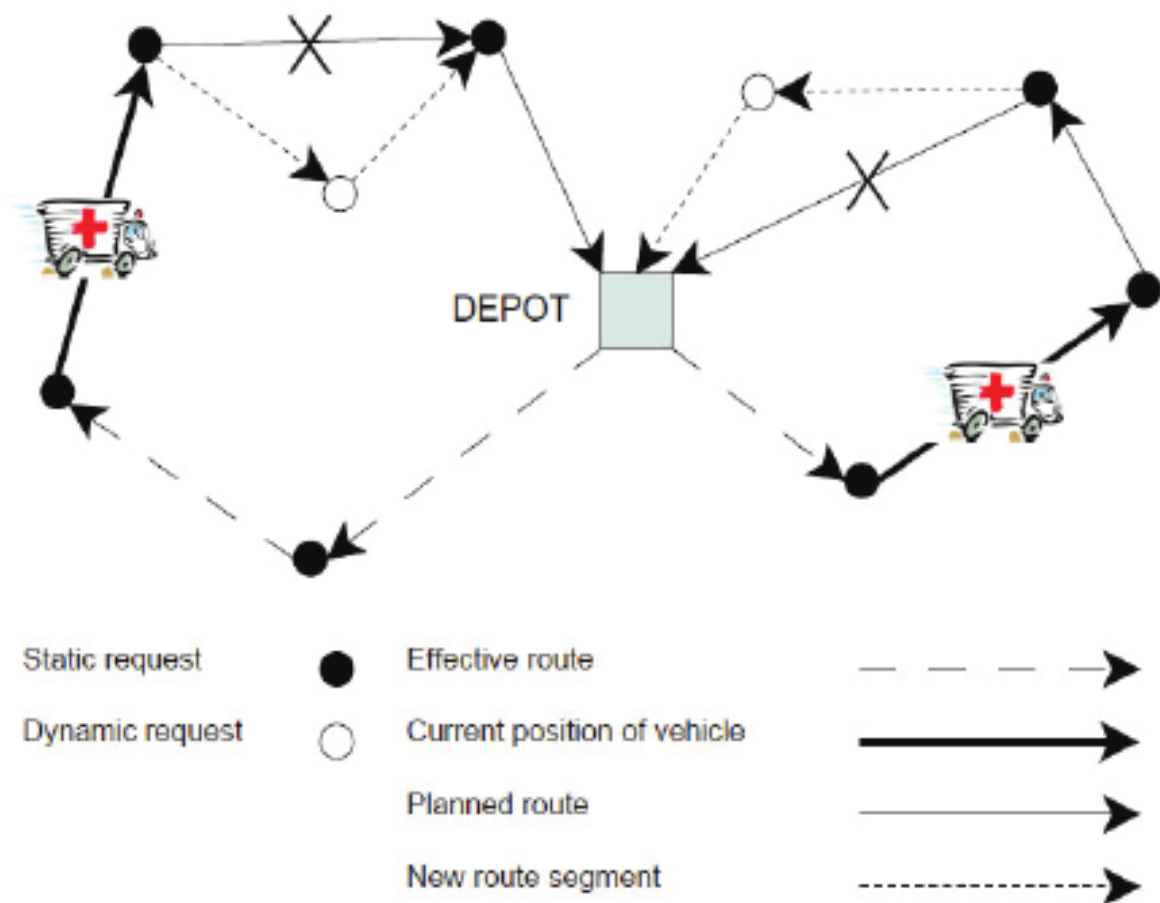
Bài toán dịch vụ chuyển phát (courier service problems): hàng hóa được thu gom tại các địa điểm khách hàng và đưa về kho trung tâm để xử lý và vận chuyển tiếp, phương tiện xuất phát rỗng. (ví dụ: Amazon)

Phân phối hàng tiêu hao (fungible/consumable goods distribution): xe được chất đầy hết hàng tại kho trước khi xuất phát và giao hàng tại các điểm khách hàng. (ví dụ: phân phối nhiên liệu cho hệ thống sưởi)

Thế nhưng, cũng không thể dự đoán được thống kê đơn hàng trong tương lai, cho nên chúng ta cần đề xuất những giải pháp để giải quyết bài toán, với các cơ chế và thuật toán.

1.3.2. Ví dụ về bài toán định tuyến xe động

Giả sử một case study thực tế là bài toán VRP động trong phân phối được phẩm, được mô tả dưới dạng hình ảnh như sau:



Hình 1.2. Ví dụ về bài toán định tuyến xe động

Hình ảnh minh họa một hệ thống giao hàng được phẩm với các thành phần cơ bản tương tự một bài toán VRP tĩnh: có một kho trung tâm (depot), có nhiều khách hàng cần phục vụ, các xe đang di chuyển trên lộ trình đã định sẵn. Thế nhưng, trong quá trình giao hàng, các nhu cầu mới xuất hiện, buộc tuyến đường phải thay đổi. Nút chấm đen biểu thị cho **yêu cầu tĩnh (Static Request)** thể hiện các đơn hàng đã lên kế hoạch từ đầu, còn nút chấm trắng biểu thị cho **yêu cầu động (Dynamic Request)** thể hiện các đơn hàng phát sinh trong lúc xe đang di chuyển. Xe chở được phẩm đang thực hiện vận chuyển và giao hàng với những mũi tên liền thể hiện **lộ trình tuyến xe đang thực hiện**, là lộ trình cập nhật với những yêu cầu mới phát sinh. Các đường nét liền mỏng chính là **lộ trình ban đầu**, còn các đường nét liền đứt chính là **lộ trình mới sau khi có phát sinh yêu cầu mới**.

Từ đó, ta có thể hiểu được cách thức hoạt động của bài toán VRP động: ban đầu xe xuất phát từ kho hàng và đã xác định được lộ trình ban đầu. Trong quá trình vận chuyển, phát sinh thêm những yêu cầu mới buộc xe phải thay đổi lộ trình và tính toán lại tuyến đường. Sau khi cập nhật, các tuyến cũ trước đó đã loại bỏ, một tuyến đường mới đã tạo ra và xe thay đổi hướng di chuyển mới để phục vụ khách hàng mới.

1.3.3. Mục tiêu của bài toán định tuyến xe động

Bài toán VRP động hướng đến việc xây dựng và duy trì một hệ thống điều phối vận chuyển hiệu quả trong môi trường thay đổi theo thời gian. Các mục tiêu chính của bài toán bao gồm:

- Tối thiểu hóa tổng quãng đường hoặc tổng chi phí vận chuyển.
- Đảm bảo tất cả khách hàng được phục vụ đầy đủ và đúng yêu cầu.
- Thỏa mãn các ràng buộc về tải trọng và thời gian phục vụ.
- Phản ứng kịp thời với các đơn hàng phát sinh trong quá trình vận hành.
- Duy trì tính khả thi và ổn định của các tuyến đã cam kết.

Như vậy, bài toán định tuyến xe động không chỉ hướng đến tối ưu chi phí mà còn phải đảm bảo khả năng thích nghi, tính khả thi và độ tin cậy trong môi trường vận hành thực tế.

1.4. Ứng dụng của bài toán định tuyến xe động

Bài toán VRP động có tính ứng dụng rất cao trong các hệ thống vận hành hiện đại, đặc biệt trong bối cảnh dữ liệu thay đổi theo thời gian thực. Không giống như VRP tĩnh, DVRP cho phép hệ thống thích nghi linh hoạt với các yêu cầu mới phát sinh, từ đó nâng cao hiệu quả vận hành và chất lượng dịch vụ, phù hợp với tình hình hiện tại của thế giới, khi mà việc giao thương giữa các quốc gia trên thế giới diễn ra liên tục và thường xuyên.

Giao hàng thương mại điện tử và logistics: bài toán VRP động có ứng dụng trong việc phân công và điều chỉnh lộ trình giao hàng theo thời gian thực, từ

đó giảm chi phí vận chuyển, tăng tốc độ giao hàng, nâng cao trải nghiệm khách hàng.

Dịch vụ giao đồ ăn và vận chuyển theo yêu cầu: bài toán VRP động có ứng dụng trong việc gán đơn hàng cho tài xế và tối ưu tuyến giao nhiều điểm, từ đó giảm thời gian chờ, tăng số đơn xử lý, tối ưu thu nhập tài xế.

Dịch vụ vận tải hành khách (ride-sharing): bài toán VRP động có ứng dụng trong việc ghép khách, định tuyến xe đón/trả nhiều điểm, từ đó tiết kiệm nhiên liệu, giảm ùn tắc, tăng hiệu suất sử dụng xe.

Vận chuyển y tế và cấp cứu: bài toán VRP động có ứng dụng trong việc điều phối xe cứu thương trong tình huống khẩn cấp, từ đó giảm thời gian cứu hộ, tăng khả năng sống sót của bệnh nhân.

Phân phối hàng hóa công nghiệp: bài toán VRP động có ứng dụng trong việc điều chỉnh kế hoạch giao hàng khi có thay đổi nhu cầu, từ đó giảm chi phí vận hành, tăng hiệu quả phân phối.

1.5. Các nghiên cứu liên quan đến bài toán định tuyến xe động

Xem xét dựa trên bối cảnh và phạm vi nghiên cứu của báo cáo, bài toán VRP động có mối liên hệ chặt chẽ với bài toán VRP. Điểm khác biệt chính là các đơn hàng mới xuất hiện khi ngày làm việc đã bắt đầu, tạo nên sự tối ưu cho bài toán. Cho nên, bài toán VRP động cũng có những ràng buộc phải thỏa mãn về khách hàng, về điểm xuất phát và kết thúc, về dung lượng xe, về thời gian phục vụ. Dựa theo báo cáo nghiên cứu, các tác giả đã liệt kê và đề xuất những thuật toán phù hợp như sau:

1.5.1. Phương pháp dựa trên thông tin thống kê [3]

Mô tả bài toán: sử dụng bài toán thuộc nhóm bài toán định tuyến xe động (DVRP), điểm khác biệt là có thêm thông tin thống kê về sự xuất hiện của đơn hàng trong tương lai (xác suất xuất hiện theo thời gian, khu vực,...)

Thuật toán áp dụng: sử dụng *thuật toán tối ưu dựa trên dự báo (stochastic/probabilistic approach)* kết hợp tổ hợp thông tin hiện tại và thông tin dự đoán. Áp dụng heuristic hoặc metaheuristic để cập nhật tuyến theo thời gian.

Quá trình giải quyết bài toán:

- Thu thập dữ liệu lịch sử, xây dựng phân bố xác suất xuất hiện đơn hàng.
- Tại mỗi thời điểm, tối ưu tuyến hiện tại, ước lượng “chi phí tương lai kỳ vọng” nếu có đơn hàng phát sinh.
- Điều chỉnh tuyến không chỉ theo hiện tại mà còn theo dự đoán.

Kết quả đạt được:

- Kết quả đạt được có hiệu suất tốt hơn so với heuristic chỉ dựa vào dữ liệu hiện tại.
- Giảm chi phí vận chuyển và số lần tái tối ưu.

Ưu điểm:

- Tận dụng được thông tin tương lai, để từ đó cải thiện chất lượng lời giải.
- Phù hợp với hệ thống logistics có dữ liệu lịch sử tốt.

Nhược điểm:

- Phụ thuộc mạnh vào độ chính xác của mô hình dự đoán.
- Tăng độ phức tạp tính toán.

1.5.2. Phương pháp mô-đun lập kế hoạch tích hợp DVRP [4]

Mô tả bài toán: sử dụng bài toán DVRP trong *hệ thống vận tải doanh nghiệp*, tập trung vào việc kế hoạch và điều phối động khi có đơn hàng mới liên tục.

Thuật toán áp dụng: sử dụng *phương pháp planning module (mô-đun lập kế hoạch)* tích hợp Dynamic VRP và các quyết định vận hành thực tế (xe, tải xé, thời gian)

Quá trình giải quyết bài toán: xây dựng một hệ thống hoạt động gồm 2 phần: **planning (offline)** để tạo tuyến ban đầu và **execution (online)** để cập nhật khi có đơn hàng mới. Khi có đơn hàng mới, chèn vào tuyến hiện tại (insertion heuristic) hoặc tái tối ưu một phần tuyến.

Kết quả đạt được: Hệ thống hoạt động hiệu quả trong môi trường thực tế có khả năng giảm thời gian phản hồi và tăng khả năng phục vụ đơn hàng động.

Ưu điểm:

- Có tính ứng dụng thực tế cao.
- Linh hoạt, phù hợp với môi trường doanh nghiệp.

Nhược điểm:

- Khó tối ưu toàn cục (do phải phản ứng nhanh).
- Phụ thuộc vào cấu trúc hệ thống và dữ liệu doanh nghiệp.

1.5.3. Phương pháp phân tích hiệu năng heuristic theo mức độ động [5]

Mô tả bài toán: sử dụng bài toán DVRP với mục tiêu nghiên cứu *mức độ động của bài toán ảnh hưởng đến hiệu suất thuật toán*. Các yếu tố độ động cần xác định gồm tần suất xuất hiện đơn hàng và thời điểm phát sinh.

Thuật toán áp dụng: sử dụng thuật toán *heuristic đơn giản*, linh hoạt thay đổi các tham số động để quan sát hiệu quả.

Quá trình giải quyết bài toán: thiết lập nhiều tình huống giả định (từ ít động cho đến nhiều động), với mỗi tình huống chạy cùng một heuristic và so sánh hiệu suất, sau đó đánh giá bằng chi phí tuyến và thời gian tính toán.

Kết quả đạt được: khi mức độ động tăng thì chất lượng lời giải giảm, thuật toán cần thích nghi tốt hơn.

Ưu điểm:

- Giúp hiểu rõ bản chất của bài toán DVRP.
- Dễ triển khai, dễ phân tích.

Nhược điểm: Thuật toán đơn giản nên chất lượng lời giải chưa cao, chỉ mang tính chất nghiên cứu yếu tố hơn là ứng dụng thực tế.

1.5.4. Thuật toán *Ant Colony Optimization (ACO)* cho bài toán động [6]

Mô tả bài toán: sử dụng bài toán *Dynamic Traveling Salesman Problem (DTSP)*, với các thành phần như khoảng cách, đỉnh hoặc chi phí có thể thay đổi theo thời gian.

Thuật toán áp dụng: sử dụng thuật toán *Ant Colony Optimization (ACO)* mô phỏng hành vi tìm kiếm thức ăn và cập nhật pheromone liên tục để thích nghi với thay đổi của loài kiến.

Quá trình giải quyết bài toán:

- Khởi tạo pheromone trên các cạnh từ một điểm bất kỳ tới điểm tiếp theo.
- Mỗi “con kiến” xây dựng tuyến đường.
- Khi môi trường thay đổi, loài kiến sẽ thực hiện giảm pheromone cũ và tăng pheromone tuyến tốt mới.
- Thực hiện lặp lại quá trình trên để thích nghi dần.

Kết quả đạt được: thích nghi tốt với thay đổi động, cho lời giải tốt hơn heuristic truyền thống.

Ưu điểm:

- Thích nghi tốt với môi trường động.
- Có khả năng tìm kiếm lời giải gần như tối ưu.

Nhược điểm:

- Tốn thời gian tính toán.
- Nhạy cảm với các tham số (alpha, beta, evaporation rate).

Trong báo cáo này, ta sẽ phân tích và triển khai thuật toán được các tác giả đề xuất là *Ant Colony System (ACS)* để xây dựng lời giải cho bài toán định tuyến xe động.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

2.1. Khái quát về thuật toán Ant Colony System (ACS)

Thuật toán **Ant Colony System (ACS)** là một phương pháp tối ưu hóa dựa trên quần thể (metaheuristic) được phát triển từ thuật toán **Tối ưu hóa đàn kiến – Ant Colony Optimization (ACO)** [7]. Thuật toán được đề xuất và cải tiến bởi nhà khoa học người Ý Luca Maria Gambardella vào năm 1996 từ thuật toán gốc ACO cũng từ một nhà khoa học người Ý khác là Marco Dorigo. Ý tưởng của thuật toán ACO bắt nguồn từ việc mô phỏng hành vi tìm kiếm thức ăn của đàn kiến trong tự nhiên, trong đó có các cá thể kiến có khả năng tìm ra đường đi ngắn nhất giữa tổ và nguồn thức ăn thông qua cơ chế trao đổi bằng pheromone. Để từ đó, thuật toán ACS đã được phát triển và thiết kế mở rộng hơn nhằm tăng tốc quá trình hội tụ và cải thiện chất lượng lời giải cho bài toán.

Như ta đã biết, thuật toán ACS được xây dựng dựa trên mô phỏng hành vi tìm kiếm thức ăn của loài kiến thông qua việc tìm kiếm đường đi ngắn nhất giữa tổ và nguồn thức ăn bằng một chất gọi là pheromone. Pheromone là một chất được tiết ra từ cơ thể loài kiến nhằm đánh dấu đường đi của chúng giữa tổ và nguồn thức ăn. Trong suốt quá trình tìm kiếm nguồn thức ăn, chúng sẽ di chuyển và để lại pheromone trên đường đi. Thời gian càng dài, pheromone mà chúng để lại sẽ giảm đi và bay hơi hoàn toàn. Đồng nghĩa với việc những con đường ngắn nhất sẽ lưu lại nồng độ pheromone cao hơn, cho nên đàn kiến sẽ di chuyển những con đường đó và có những phản hồi tích cực dẫn đến việc các con kiến sau cũng sẽ đi một con đường duy nhất. Từ đó chúng tìm thấy được con đường tốt nhất tới nguồn thức ăn từ tổ của chúng,

Đối với bài toán định tuyến xe động, thuật toán được phát triển và hoạt động theo cách **tập trung (centralized)**. Trong bài toán, chúng ta sẽ gọi giai đoạn mà một đơn hàng được thông báo cho tài xế là **giai đoạn cam kết (commitment phase)**, tức là một khi đơn hàng bất kỳ đã gán cho tài xế thì sẽ không thể thay đổi. Đồng thời, trong phương pháp này, các tác giả đã có ý tưởng chia ngày làm việc

thành **nts** khoảng thời gian (time-slices) có độ dài là $\frac{T}{nts}$, trong đó **T** là độ dài ngày làm việc bao gồm cả khoảng thời gian trì hoãn việc xử lý mỗi đơn hàng phát sinh [8]. Ý tưởng này xuất phát từ việc các phương tiện có tải trọng và vị trí xuất phát không đồng nhất, nhằm giới hạn thời gian dành cho mỗi bài toán VRP. Một nhược điểm của việc chia khoảng thời gian là thời gian dành cho mỗi bài toán VRP không thể biết trước, dẫn đến kết quả không hợp lý và thỏa đáng. Các tham số khác như **thời điểm cắt (cut-off time)** và **thời gian cam kết trước (advanced commitment time)** cũng áp dụng trong thuật toán để lọc các đơn hàng và đảm bảo các tài xế có đủ thời gian phản ứng khi nhận các đơn hàng mới.

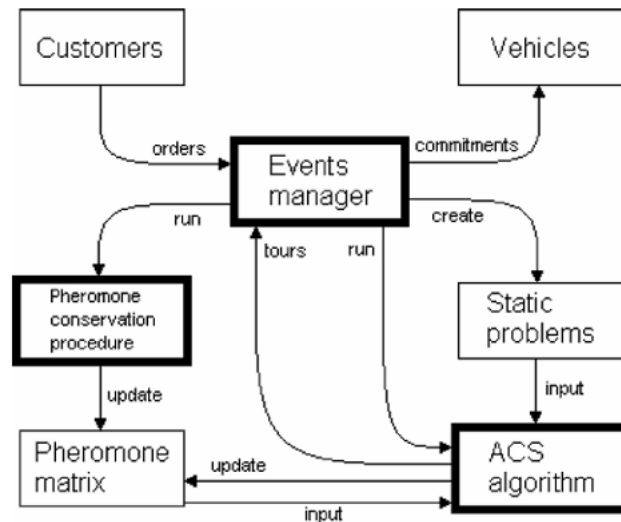
2.2. Mô hình hóa của bài toán định tuyến xe động

Theo như chúng ta đã tìm hiểu ở chương 1, bài toán định tuyến xe động (Dynamic Vehicle Routing Problem) có mối liên hệ chặt chẽ với bài toán VRP truyền thống. Có thể nói, bài toán VRP động được mô hình hóa như một chuỗi các bài toán VRP. Cụ thể, với mỗi bài toán VRP tại một thời điểm sẽ bao gồm tất cả các khách hàng đã biết tại thời điểm đó nhưng chưa được phục vụ. Áp dụng thuật toán ACS, bài toán định tuyến xe động được mô hình hóa dưới dạng kiến trúc hệ thống bao gồm ba thành phần chính:

Bộ quản lý sự kiện (Events manager): thu thập các đơn hàng mới, theo dõi các đơn đã phục vụ và vị trí của các phương tiện, đồng thời chịu trách nhiệm gán đơn hàng cho tài xế.

Thuật toán ACS (Ant Colony System): được sử dụng để giải quyết chuỗi các bài toán VRP được tạo ra bởi bộ quản lý sự kiện dạng heuristic.

Thủ tục bảo toàn pheromone (Pheromone conservation procedure): là một thành phần quan trọng trong kiến trúc, giúp truyền tải hiệu quả thông tin về các đặc điểm của các lời giải tốt từ một khoảng thời gian bất kỳ sang khoảng thời gian tiếp theo.



Hình 2.1. Mô hình hóa của bài toán định tuyến xe động

2.3. Phân tích và thiết kế bộ quản lý sự kiện (Events manager)

2.3.1. Vai trò và chức năng của bộ quản lý sự kiện

Bộ quản lý sự kiện (Events manager) là thành phần trung gian đóng vai trò kết nối giữa hệ thống tối ưu và môi trường thực tế. Thành phần này chịu trách nhiệm trong việc:

- Tiếp nhận các đơn hàng mới từ khách hàng.
- Theo dõi trạng thái hoạt động của các phương tiện.
- Xây dựng các bài toán VRP theo từng thời điểm.
- Điều phối việc gán (cam kết) đơn hàng cho các phương tiện.

Nói cách khác, bộ quản lý sự kiện đóng vai trò như một bộ điều phối thông minh (dispatcher) trong hệ thống bài toán VRP động.

2.3.2. Dữ liệu đầu vào và đầu ra

a. Dữ liệu đầu vào

- Danh sách đơn hàng: bao gồm đơn hàng của ngày làm việc trước đó và phát sinh trong ngày.

- Trạng thái phương tiện: vị trí hiện tại, tải trọng còn lại và thời gian sẵn sàng.

- Các tham số hệ thống:
- T_{co} (cut-off time) là thời điểm ngừng nhận đơn.
- T_{ac} (advanced commitment time) là thời gian cam kết trước.
- nts là số khoảng thời gian.

b. Dữ liệu đầu ra

- Danh sách các đơn hàng đã cam kết cho từng xe.
- Lộ trình cập nhật của các phương tiện.
- Trạng thái hệ thống sau mỗi khoảng thời gian.

2.3.3. Nguyên lý hoạt động của bộ quản lý sự kiện

Bộ quản lý sự kiện hoạt động dựa theo những cơ chế và quy tắc như sau:

- Nguyên tắc chia thời gian: như ta đã đề cập trước đó, trong phạm vi của thuật toán ACS đối với bài toán định tuyến xe động, ngày làm việc sẽ **chia thành các khoảng thời gian bằng nhau (time-slices)** để hệ thống tạm thời tích lũy các đơn hàng mới.

- Nguyên tắc tạo bài toán VRP: tại mỗi đầu khoảng thời gian, xây dựng một bài toán VRP gồm các đơn hàng đã biết nhưng chưa được cam kết, xe sẽ bắt đầu thực hiện bài toán đó tại **vị trí khách hàng cuối phục vụ và tải trọng còn lại của xe**.

- Nguyên tắc tối ưu hóa: được giải quyết bằng **cơ chế bảo toàn pheromone** và **thuật toán ACS**, từ đó ta được **một tập lời giải (routes)**.

- Nguyên tắc cam kết đơn hàng: xét các đơn hàng có thời gian phục vụ trong khoảng $T_{nts} + T_{ac}$, các đơn đã cam kết sẽ không thay đổi còn những đơn chưa cam kết sẽ được tối ưu dựa trên điều kiện **thời điểm bắt đầu phục vụ nằm trong khoảng thời gian đó và thỏa mãn $\geq T_{ac}$** .

Xét nếu có trường hợp đặc biệt, một phương tiện được phép quay về kho khi thỏa mãn các điều kiện sau:

- Thời gian hiện tại $\geq T_{CO}$ và tất cả khách hàng đã được phục vụ.
- Xe đã sử dụng hết tải trọng.

Nếu phương tiện đó không thỏa mãn hai điều kiện trên thì buộc phương tiện sẽ chờ tại khách hàng cuối cùng và có thể phát sinh các đơn hàng mới buộc lộ trình phải cập nhật và tối ưu lại.

2.3.4. Mã giả của bộ quản lý sự kiện

Mã giả (pseudo-code) là cách mô tả thuật toán bằng ngôn ngữ gần với con người, không phụ thuộc vào bất kỳ ngôn ngữ lập trình nào. Xét trong tài liệu nghiên cứu “*Ant Colony System for a Dynamic Vehicle Routing Problem*”, sau khi đã hiểu sâu bản chất cốt lõi của mô hình, các tác giả đã phân tích và triển khai phân mã giả cho bộ quản lý sự kiện như sau:

Dữ liệu đầu vào (Inputs):

- Kho hàng (depot): (x_0, y_0)
- Đội xe: gồm tập hợp xe $V = \{v_1, v_2, \dots, v_m\}$, sức chứa mỗi xe Q .
- Tập đơn hàng động: $PendOrds$ (thời điểm bắt đầu) với mỗi đơn hàng i .
- Ma trận khoảng cách $D = [d_{ij}]$, các tham số thời gian $T, T_{ts}, n_{ts}, T_{ac}$.
- Tham số thuật toán ACS với ma trận pheromone τ_{ij} với các tham số α, β, ρ, q_0 .

Dữ liệu đầu ra (Outputs):

- Tuyến đường cho từng xe: $Route_k = (Depot \rightarrow i_1 \rightarrow i_2 \rightarrow \dots \rightarrow Depot)$
- Thời gian phục vụ cho mỗi đơn hàng.
- Trạng thái cuối cùng của hệ thống.
- Các chỉ số đánh giá đặc biệt khác.

$Time := 0;$

Điểm xuất phát của mỗi xe đều thiết lập bắt đầu từ kho hàng (depot);

$PendOrds :=$ các đơn hàng đã biết từ ngày hôm trước;

While ($PendOrds \neq \emptyset$ or $Time < T_{co}$)

If ($Time > 0$)

$StaticProb :=$ bài toán gồm các đơn hàng trong $PendOrds$ và thời gian bắt đầu $Time + T_{ac}$;

Else

$StaticProb :=$ bài toán gồm các đơn hàng trong $PendOrds$ và thời gian bắt đầu $Time$;

EndIf

 Chạy thuật toán ACS trên $StaticProb$;

$CommOrds :=$ các đơn hàng có thời gian xử lý $\geq Time + \frac{T}{n_{ts}} + T_{ac}$

 Cam kết (commit) các đơn hàng trong $CommOrds$;

$PendOrds := PendOrds \setminus CommOrds$;

$PendOrds := PendOrds \cup \{\text{các đơn hàng xuất hiện trong } \frac{T}{n_{ts}} \text{ giây gần nhất}\};$

$Time := Time + T_{ts}$;

 Cập nhật vị trí bắt đầu, sức chứa của xe và thời gian di chuyển;

 Cập nhật ma trận pheromone;

EndWhile

Cam kết *depot* là điểm đến cuối cùng cho tất cả các xe;

Bộ quản lý sự kiện hoạt động theo dạng mã giả có nhiệm vụ như sau:

- Xác định bài toán VRP trong mỗi khoảng thời gian.
- Giải bài toán VRP bằng thuật toán ACS.
- Cam kết các đơn hàng có thể phục vụ chắc chắn trong khoảng thời gian đó.

- Cập nhật hệ thống (vị trí xe, sức chứa, thời gian di chuyển, ma trận pheromone).
- Lặp lại cho đến khi không còn đơn hàng chờ hoặc vượt quá thời gian kết thúc T_{CO} .

Dựa theo mã giả trên của bộ quản lý sự kiện, ta xác định bộ quản lý xử kiện được vận hành với quy trình như sau:

- **Khởi tạo:** bắt đầu xuất phát tại *depot*, với các đơn hàng đã biết từ hôm trước (*PendOrds*) và $Time := 0$.

- **Thực hiện vòng lặp:** giải bài toán VRP bắt đầu từ cửa sổ thời gian xét theo điều kiện của $Time$ ($Time = 0$ hoặc $Time > 0$)

- Giải bài toán bằng **thuật toán ACS** để tìm tuyến tối ưu.

- **Thực hiện cam kết:** các đơn hàng phải được xử lý trước mốc thời gian $Time + \frac{T}{n_{ts}} + T_{ac}$.

- Thực hiện loại bỏ các đơn hàng đã cam kết khỏi danh sách chờ (*PendOrds*) và thêm các đơn hàng mới xuất hiện trong $\frac{T}{n_{ts}}$ gần nhất.

- Tăng thêm thời gian bằng T_{ts} , cập nhật trạng thái xe và thông tin liên quan.

- Cập nhật ma trận pheromone để hỗ trợ các lần giải tiếp theo.

- **Kết thúc:** khi không còn đơn hàng chờ hoặc đã quá thời gian kết thúc T_{CO} , *depot* được cam kết là điểm cuối cùng cho tất cả các xe.

2.4. Phân tích và thiết kế thuật toán Ant Colony System

Như ta đã biết trước đó, thuật toán Ant Colony System (ACS) là một biến thể nâng cao từ thuật toán Ant Colony Optimization (ACO), dựa trên mô hình tính toán lấy cảm hứng từ hành vi của đàn kiến trong tự nhiên. Ý tưởng chính để bắt đầu thiết kế và xây dựng thuật toán là song song hóa quá trình tìm kiếm thông qua nhiều tiến trình xây dựng lời giải. Các lời giải tốt sẽ được “ghi nhớ” thông qua

pheromone. Để từ đó, ta xây dựng một lộ trình phục vụ khách hàng và từng bước chọn khách hàng tiếp theo cho đến khi hoàn thành tuyến.

2.4.1. Nguyên lý hoạt động của thuật toán Ant Colony System

Giả định một case study như sau: ta có v kho giả (dummy depots), mỗi xe tương ứng một kho, ký hiệu là $d_1, d_2, \dots, d_v$. Các lời giải do kiến tạo ra được biểu diễn dạng một chuỗi tuyến dài duy nhất, trong đó các nút nằm giữa hai kho giả liên tiếp d_a và d_b tạo thành tuyến của xe a và tuyến của xe b bắt đầu từ kho giả d_b . Kho giả d_b tương ứng với vị trí khách hàng cuối cùng phục vụ, thời gian kết thúc phục vụ khách hàng đó và tải trọng còn lại. Thành phần chính của thuật toán là kiến (ants) sẽ xây dựng lời giải bằng cách tiết pheromone trên đường đi của chúng một cách đơn giản và lặp lại. Tại mỗi bước, mỗi kiến k xác định tập các khách hàng khả thi và chọn theo xác suất p_{ij}^k để đi từ khách hàng i đến j phụ thuộc vào: độ hấp dẫn heuristic $\eta_{ij} = \frac{1}{tt_{ij}}$ và pheromone τ_{ij} .

$$p_{ij}^k = \begin{cases} \frac{\tau_{ij}(\eta_{ij})^\beta}{\sum_{l \in F_i^k} \tau_{il}(\eta_{il})^\beta} & j \in F_i^k \\ 0 & \text{otherwise} \end{cases}$$

trong đó:

- F_i^k : tập khách hàng khả thi
- β : hệ số điều chỉnh

Xác suất trên là sự cân bằng từ thông tin heuristic (a priori) và kinh nghiệm từ quá khứ (pheromone). Lúc này, kiến sẽ thực hiện tìm kiếm khách hàng j với hình thức hoạt động: **khai thác (exploitation)** với $\arg \max p_{il}^k$ với xác suất q và **khám phá (exploration)** với xác suất $1 - q$.

2.4.2. Cơ chế cập nhật pheromone

Trong thuật toán ACS, nhà khoa học Luca Maria Gambardella đã cải tiến và cập nhật pheromone theo hai cơ chế chính: ***cập nhật cục bộ (local update)*** và ***cập nhật toàn cục (global update)***.

a. Cập nhật cục bộ (local update)

Cập nhật cục bộ được thực hiện khi một con kiến di chuyển qua một cạnh với công thức như sau:

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \rho\tau_0$$

trong đó:

- ρ là hệ số bay hơi pheromone.
- τ_0 là giá trị pheromone khởi tạo.

Ưu điểm của cập nhật cục bộ là giảm sự phụ thuộc quá mức vào một tuyến đường và khuyến khích khám phá các tuyến khác.

b. Cập nhật toàn cục (global update)

Cập nhật toàn cục được thực hiện sau khi tất cả các con kiến hoàn thành việc xây dựng lời giải, thuật toán sẽ chọn ra lời giải tốt nhất và cập nhật pheromone theo công thức sau:

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \rho\Delta\tau_{ij}$$

trong đó:

$$\Delta\tau_{ij} = \begin{cases} \frac{1}{L_{best}} & \text{nếu cạnh thuộc lời giải tốt nhất} \\ 0 & \text{ngược lại} \end{cases}$$

Ưu điểm của cập nhật toàn cục là tăng cường các tuyến đường tốt và hướng quá trình tìm kiếm đến lời giải tối ưu.

2.4.3. Mã giả của thuật toán Ant Colony System

Tương tự, trong tài liệu nghiên cứu, từ mô hình và nguyên lý hoạt động của thuật toán Ant Colony System, các tác giả đã xây dựng và triển khai mã giả cho thuật toán như sau:

$BestCost := \infty;$

Với mỗi cạnh (i, j) :

$\tau_{ij} := \tau_0;$

EndFor

While (thời gian tính toán $< T_{nts}$)

For $k := 1$ to m

While (kiến k chưa hoàn thành lời giải)

Chọn khách hàng tiếp theo j ;

Cập nhật lượng pheromone trên cạnh τ_{ij} (theo công thức cập nhật cục bộ);

EndWhile

Thực hiện tìm kiếm cục bộ (thời gian tính toán tối đa $= t_{ls}$);

$Cost :=$ chi phí lời giải hiện tại;

If ($Cost < CostBest$)

$CostBest := Cost;$

$BestSol :=$ lời giải hiện tại;

EndIf

EndFor

Với mỗi cạnh (i, j) trong lời giải $BestSol$

Cập nhật lượng pheromone trên cạnh τ_{ij} (theo công thức cập nhật toàn cục);

EndFor

EndWhile

- Tìm tuyến đường tối ưu
- Học dần từ các lời giải tốt trước đó
- Cân bằng giữa khám phá (exploration) và khai thác (exploitation).

Dựa theo mã giả trên của bộ quản lý sự kiện, ta xác định bộ quản lý xử kiện được vận hành với quy trình như sau:

- **Khởi tạo:** tất cả các đường đi đều có pheromone như nhau, chưa có ưu tiên.

- **Xây dựng lời giải:** xuất phát từ *depot*, chọn khách hàng tiếp theo dựa vào pheromone và heuristic.

- **Cập nhật cục bộ (local update):** xảy ra khi kiến đi qua cạnh (i, j) , làm giảm pheromone tạm thời.

- Thực hiện **tìm kiếm cục bộ (local search)** tìm kiếm và so sánh lời giải tốt nhất với thời gian tối đa t_{ls} .

- **Cập nhật toàn cục (global update):** áp dụng với lời giải tốt nhất và tăng pheromone cho lời giải đó.

2.4.4. Ví dụ về cơ chế hoạt động của thuật toán Ant Colony System

Dựa theo phần mã giả của thuật toán Ant Colony System đã phân tích và nghiên cứu sâu trước đó, chúng em tiến hành thực nghiệm với một ví dụ thực hiện bằng tính toán đơn thuần nhằm để hiểu rõ hơn cơ chế hoạt động của thuật toán trong bài toán định tuyến xe động như sau:

$BestCost := \infty;$

Với mỗi cạnh (i, j) :

$\tau_{ij} := \tau_0 = 0.1;$

Bắt đầu vòng lặp While (thời gian tính toán $< T_{nts}$)

For $k := 1$ to m ($m = 2$)

While (kiến $k = 1$ chưa hoàn thành lời giải)

Tính xác suất để chọn khách hàng tiếp theo:

Từ kho hàng (0, 0) đến 1: $0.1 * (\frac{1}{2})^2 = 0.025$

Tương tự tính từ 0 cho đến 2, 3, dựa theo kết quả ta chọn 1.

Cập nhật lượng pheromone trên cạnh $\tau_{01} = (1 - 0.1) * 0.1 + 0.1 * 0.1 = 0.1;$

Từ 1 ta tính xác suất để chọn khách hàng tiếp theo:

Từ 1 đến 2: $0.1 * (\frac{1}{3})^2 \approx 0.011$

Tương tự tính từ 1 đến 3, dựa theo kết quả ta chọn 2.

Cập nhật lượng pheromone trên cạnh $\tau_{12} = 0.1.$

Tương tự từ 2 tính xác suất để chọn khách hàng tiếp theo.

Kết thúc vòng lặp While.

$Cost = 2 + 3 + 2 + 6 = 13$

Thực hiện tìm kiếm cục bộ, giữ nguyên giá trị $Cost$.

If ($Cost < CostBest$)

$CostBest = 13;$

$BestSol := (0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0);$

EndIf

Thực hiện tiếp vòng lặp While với $k = 2$

Tiến hành thực hiện cập nhật toàn cục với mỗi cạnh (i, j) : $\rho = 0.1, \frac{1}{13} \approx 0.077.$

Ví dụ như cạnh $\tau_{01} = 0.9 * 0.1 + 0.1 * 0.077 = 0.0977$

Tương tự thực hiện với các cạnh còn lại.

Kết thúc bài toán.

Kết quả đạt được:

- Các cạnh thuộc *BestSol* có pheromone cao hơn.
- Cạnh không tốt giảm pheromone (từ 0.1 giảm xuống 0.09).
- Hệ thống cân bằng với exploration vs exploitation.
- Sau nhiều vòng, pheromone hội tụ vào tuyến tốt nhất, thuật toán dần tìm lời giải tối ưu gần đúng.

2.5. Phân tích và thiết kế thủ tục bảo toàn pheromone (Pheromone conservation)

Sau khi đã áp dụng và thực hiện thuật toán ACS trong bài toán định tuyến xe động, các bài toán VRP ở các time-slices liên tiếp có sự tương đồng, cho nên không cần tối ưu lại từ đầu. Trong khi đó, pheromone đã chứa đựng thông tin dưới dạng mã hóa về các lời giải. Nó đã lưu trữ những thông tin liên quan đến tuyến đường tốt và giá trị các cặp khách hàng nên đi liên tiếp. Cho nên, thay vì reset toàn bộ ma trận pheromone, hệ thống sẽ giữ lại một phần thông tin cũ dưới dạng công thức:

$$\tau_{ij} = (1 - \gamma_r)\tau_{ij}^{old} + \gamma_r\tau_0$$

trong đó:

- γ_r nhỏ: giữ nhiều thông tin cũ.
- γ_r lớn: khám phá thêm thông tin mới.

Qua cơ chế này, ta không cần phải tối ưu lại từ đầu và tăng tốc độ hội tụ.

Cho nên, ta có thể nói thủ tục bảo toàn pheromone là cầu nối giữa các khoảng thời gian, giúp tăng hiệu quả và giảm chi phí tính toán, đóng vai trò vô cùng quan trọng trong toàn bộ thuật toán.

CHƯƠNG 3: CÀI ĐẶT VÀ THỰC NGHIỆM ĐÁNH GIÁ

Trước đó, trong Chương 2, ta đã tìm hiểu và phân tích hoạt động của thuật toán ACS trong bài toán định tuyến xe động, đồng thời đã hiểu bản chất và cách thức hoạt động của từng thành phần trong môi trường hệ thống của bài toán. Ở chương này, dựa vào kiến thức đã học về lập trình trước đó và công cụ có sẵn, chúng em sẽ tiến hành cài đặt chương trình hệ thống và đánh giá thực nghiệm.

3.1. Thiết lập bộ dữ liệu của bài toán định tuyến xe động

Trong phạm vi tài liệu nghiên cứu, các tác giả đã sử dụng các bài toán động được đề cập lần đầu trong một tài liệu của Kilby và cộng sự của ông [5]. Chúng được xây dựng dựa trên các bộ dữ liệu các bài toán VRP tĩnh nổi tiếng, bao gồm: 12 bài toán từ Taillard (1994) [7], 7 bài toán từ Christofides & Beasley (1984) [8] và 2 bài toán từ Fisher và cộng sự (1981) [9]. Tổng cộng bài toán dựa trên phạm vi khách hàng với số lượng từ 50 đến 199 người. Như đã phân tích trước đó, bài toán VRP động là một chuỗi gồm các bài toán VRP. Để chuyển hóa sang dạng bài toán định tuyến xe động, Kilby và cộng sự đã bổ sung thêm các đặc điểm sau:

- **Thời gian làm việc T :** là giới hạn toàn bộ bài toán, quy định thời điểm hoạt động của mọi thành phần trong hệ thống.
- **Thời điểm xuất hiện đơn hàng:** là thời điểm các đơn hàng mới phát sinh và xuất hiện dần theo các thời gian.
- **Thời gian phục vụ đơn hàng:** là khoảng thời gian cần thiết để hoàn thành việc phục vụ của mỗi khách hàng.
- **Số lượng phương tiện:** là tổng số lượng có sẵn trong hệ thống để phục vụ khách hàng, trong các bài toán trên đều đặt với số lượng là 50 xe.

Ngoài ra, Kilby và cộng sự còn quan tâm tới sự ảnh hưởng của hai tham số: thời điểm cắt T_{CO} (đơn hàng sau thời điểm này sẽ chuyển sang ngày hôm sau) và thời gian cam kết trước T_{ac} (đơn phải gán trước cho xe trước bao lâu). Dựa theo nền tảng và số liệu đã rút ra được từ những công trình nghiên cứu và thử nghiệm, trong phạm vi nghiên cứu này, các tác giả đã thiết lập tham số như sau: $T_{CO} =$

$0.5T$ và $T_{ac} = 0$. Việc điều chỉnh tham số có ý nghĩa: giúp bài toán tránh sự “động”, tạo cột mốc rõ ràng để đánh giá, có thể linh hoạt thay đổi tuyến.

Đồng thời, trong phạm vi nghiên cứu này, các tác giả đã điều chỉnh ánh xạ thời gian làm việc T tương đương 1500 giây CPU. Trong thực tế, thuật toán ACS áp dụng trong bài toán định tuyến xe động sẽ chạy trong suốt một ngày làm việc. Cho nên, thời gian xuất hiện đơn hàng và thời gian phục vụ đều sẽ chuẩn hóa theo tỷ lệ tương ứng.

3.2. Thiết lập tham số của thuật toán ACS

Trong phạm vi nghiên cứu, các tác giả dựa theo nghiên cứu trước đó của Gambardella và cộng sự [10] để tiến hành hiệu chỉnh một số tham số trong khuôn khổ thuật toán ACS cho bài toán định tuyến xe động. Các tham số này được sử dụng trong việc giải quyết các bài toán VRP tĩnh sinh ra trong quá trình xử lý động. Theo nghiên cứu của Gambardella và cộng sự, một bộ tham số tốt cho thuật toán ACS khi áp dụng cho VRP cổ điển bao gồm các giá trị như sau:

- $q_0 = 0.9, \beta = 1, \rho = 0.1$
- Một số lượng nhỏ đàn kiến, ví dụ $m = 3$.
- $\tau_0 = \frac{1}{n \times Cost(PI)}$, trong đó $Cost(PI)$ là chi phí lời giải thu được bằng thuật toán tham lam.

Một trong những tham số quan trọng trong bài toán định tuyến xe động là thời gian tìm kiếm cục bộ t_{ls} . Trong phạm vi nghiên cứu, để hiệu chỉnh cẩn thận tham số này, các tác giả đã thiết lập tham số $t_{ls} = \frac{T}{6.n_{ts}}$. Do đó, tham số n_{ts} là tham số rất quan trọng, cần phải hiệu chỉnh cẩn thận bởi nếu quá lớn, thuật toán sẽ phải khởi động lại quá thường xuyên, không đạt được cực tiểu cục bộ tốt. Còn nếu quá nhỏ, thuật toán tối ưu lâu trên dữ liệu cũ, không cập nhật thông tin mới.

Cho nên, các tác giả đã tiến hành thực nghiệm và đánh giá với các tình huống giả định n_{ts} lần lượt với 10, 25, 50. Đồng thời, các tác giả đã thiết lập cố định tham số bảo toàn pheromone $\gamma_r = 0.3$ dựa theo đề xuất của các đánh giá trước

đó. Sau đó tiến hành chạy lần lượt trên 3 bộ dữ liệu **c100**, **f71** và **tai75a** của bài toán. Ba bộ dữ liệu này được lấy từ thư viện benchmark **TSPLIB**, thường được dùng để đánh giá hiệu suất tối ưu của thuật toán cho bài toán và so sánh hiệu năng giữa các phương pháp. Dưới đây các nhà nghiên cứu tiến hành chạy thử nghiệm các bộ dữ liệu, mỗi tình huống chạy 5 lần. Kết quả được ghi nhận dưới bảng sau, xét xu hướng chung khi thay đổi n_{ts} và độ ổn định của thuật toán, được đánh giá dựa theo các tham số: *Min* (tốt nhất), *Max* (tệ nhất) và *Arg* (trung bình), cho thấy $n_{ts} = 25$ đạt hiệu suất tốt nhất, cho thấy sự cân bằng giữa khả năng khai thác và khám phá của thuật toán:

Table 1. Calibration of parameter n_{ts} (number of time slices).

n_{ts}	$\frac{T}{n_{ts}}$	t_{ls}		c100	f71	tai75a
10	150	25	Min	1004.58	311.95	1880.11
			Max	1145.20	399.26	2105.14
			Avg	1083.64	362.93	1963.19
25	60	10	Min	973.26	311.18	1843.08
			Max	1100.61	420.14	2043.82
			Avg	1066.16	348.69	1945.20
50	30	5	Min	1131.95	333.25	1966.92
			Max	1228.97	452.73	2133.87
			Avg	1185.25	417.74	2019.82

Hình 3.1. Thí nghiệm đánh giá tham số n_{ts}

Tương tự, tham số bảo toàn pheromone γ_r cũng được các tác giả tiến hành với các thử nghiệm tham số lần lượt là 0.1, 0.3, 0.5, 1.0 qua bảng sau, cho thấy 0.3 là tham số cho kết quả tốt nhất. Với giá trị tham số quá nhỏ, pheromone bay hơi nhanh làm giảm khả năng khai thác thông tin, trong khi giá trị quá lớn khiến thuật toán dễ hội tụ sớm vào nghiệm cục bộ. Cho nên, 0.3 là lựa chọn giá trị cân bằng cho hành động khám phá và khai thác của thuật toán:

Table 2. Tuning of γ_r (parameter for pheromone conservation).

γ_r		c100	f71	tai75a
0.1	Min	1072.44	352.77	1928.18
	Max	1157.43	419.21	2220.75
	Avg	1116.13	383.27	2016.78
0.3	Min	973.26	311.18	1843.08
	Max	1100.61	420.14	2043.82
	Avg	1066.16	348.69	1945.20
0.5	Min	1039.36	360.20	1847.41
	Max	1135.26	443.16	2054.91
	Avg	1087.90	389.00	1962.66
1.0	Min	1079.12	367.32	1873.69
	Max	1133.15	431.53	2102.58
	Avg	1098.99	399.18	1971.91

Hình 3.2. Thí nghiệm đánh giá tham số γ_r

3.3. Đánh giá thuật toán ACS với các thuật toán khác

3.3.1. Khái quát về thuật toán Greedy Randomized Adaptive Search Procedure (GRASP)

Để đánh giá hiệu quả của thuật toán ACS trong bài toán định tuyến xe động, các tác giả đã tiến hành đánh giá so sánh với **thuật toán tìm kiếm thích nghi tham lam ngẫu nhiên** – hay còn gọi là **thuật toán GRASP**. Thuật toán lần đầu xuất hiện trong một nghiên cứu của Resende và Ribeiro (2003) [11]. Thuật toán này có kiến trúc tương tự như thuật toán ACS khi áp dụng trong bài toán định tuyến xe động, nhưng có hai điểm khác biệt chính: sử dụng GRASP làm điểm khởi đầu thay vì thuật toán ACS và không có cơ chế pheromone, kết hợp với yếu tố tương như mâu thuẫn từ thuật toán tham lam (greedy). Dựa theo nghiên cứu, thuật toán GRASP trong bài toán định tuyến xe động hoạt động theo cơ chế trong mỗi khoảng thời gian (time slice) như sau:

Xây dựng lời giải ban đầu: khách hàng tiếp theo được chọn ngẫu nhiên trong tập với công thức như sau:

$$[tt_{min}, tt_{min} + \delta_G(tt_{max} - tt_{min})]$$

trong đó:

- tt_{min}, tt_{max} : thời gian di chuyển nhỏ nhất và lớn nhất khả thi.
- δ_G : tham số điều khiển mức độ ngẫu nhiên.

Cơ chế này sẽ lặp lại cho đến khi xây dựng lời giải hợp lý. Các tác giả nhận ra rằng khi tham số $\delta_G = 0$ nó sẽ hoàn toàn greedy, còn $\delta_G = 1$ nó sẽ hoàn toàn ngẫu nhiên. Cho nên, trong phạm vi nghiên cứu, các tác giả sẽ thiết lập tham số $\delta_G = 0.75$.

Tìm kiếm cục bộ: áp dụng tương tự như thuật toán ACS, chạy trong thời gian t_{ls} .

Cập nhật lời giải tốt nhất: nếu lời giải mới tốt hơn sẽ tiến hành cập nhật.

Thuật toán GRASP thường được ứng dụng thực tế trong các bài toán như bài toán người du lịch (TSP), bài toán định tuyến xe (VRP), lập lịch (scheduling), phân cụm (clustering),...

3.3.2. Đánh giá thực nghiệm giữa thuật toán ACS và thuật toán GRASP trong bài toán định tuyến xe động

Trong phạm vi nghiên cứu, các tác giả đã tiến hành thực nghiệm so sánh giữa thuật toán ACS và thuật toán GRASP trong bài toán định tuyến xe động. Các bộ dữ liệu sử dụng được lấy từ nguồn thư viện **TSPLIB** và một phần từ nhóm dữ liệu **Taillard (*tai)**. Chúng thường được dùng liên quan đến các bài toán định tuyến tối ưu, nhưng trong phạm vi nghiên cứu này, các tác giả sử dụng các bộ dữ liệu trên để so sánh hiệu năng giữa hai thuật toán và đảm bảo tính tái lập kết quả nghiên cứu. Tương tự như đánh giá và thực nghiệm với các tham số trước đó, các chỉ số đánh giá sẽ bao gồm: *Min*, *Max* và *Arg*. Kết quả đạt được thể hiện dưới bảng sau, cho thấy hiệu suất của thuật toán ACS trong bài toán định tuyến tốt hơn so với thuật toán GRASP trong 5 lần chạy. Thuật toán GRASP cho nghiệm ban đầu đạt giá trị tốt, nhưng quá phụ thuộc vào tìm kiếm cục bộ, khó đưa ra giá trị lời giải tốt nhất. Trong khi đó, nhờ cơ chế pheromone, qua việc hướng dẫn tìm kiếm và cải thiện chất lượng nghiệm giúp thuật toán ACS có khả năng mở rộng đối với các bài toán có kích thước lớn hơn:

Table 3. Computational results.

Problem	GRASP-DVRP			ACS-DVRP		
	Min	Max	Avg	Min	Max	Avg
c100	1080.33	1169.67	1119.06	973.26	1100.61	1066.16
c100b	978.39	1173.01	1022.12	944.23	1123.52	1023.60
c120	1546.50	1754.00	1643.15	1416.45	1622.12	1525.15
c150	1468.36	1541.54	1501.35	1345.73	1522.45	1455.50
c199	1774.33	1956.76	1898.20	1771.04	1998.87	1844.82
c50	696.92	757.97	719.56	631.30	756.17	681.86
c75	1066.59	1142.32	1079.16	1009.38	1086.65	1042.39
f134	15433.84	17325.73	16458.47	15135.51	17305.69	16083.56
f71	359.16	429.64	376.66	311.18	420.14	348.69
tai100a	2427.07	2583.02	2510.29	2375.92	2575.70	2428.38
tai100b	2302.95	2636.05	2512.27	2283.97	2455.55	2347.90
tai100c	1599.19	1800.85	1704.40	1562.30	1804.20	1655.91
tai100d	1973.03	2165.39	2087.55	2008.13	2141.67	2060.72
tai150a	3787.53	4165.42	3899.16	3644.78	4214.00	3840.18
tai150b	3313.03	3655.63	3485.79	3166.88	3451.69	3327.47
tai150c	3110.10	3635.17	3219.27	2811.48	3226.73	3016.14
tai150d	3159.21	3541.27	3298.76	3058.87	3382.73	3203.75
tai75a	1911.48	2116.95	2005.44	1843.08	2043.82	1945.20
tai75b	1582.24	1934.35	1758.88	1535.43	1923.64	1704.06
tai75c	1596.17	1859.71	1674.37	1574.98	1842.42	1653.58
tai75d	1545.21	1641.91	1588.73	1472.35	1647.15	1529.00
Total	52731.63	58986.36	55562.64	50855.94	57645.52	53784.02

Hình 3.3. Thí nghiệm so sánh thuật toán

3.4. Môi trường phát triển

Trong quá trình thực hiện đồ án, nhóm đã sử dụng các công nghệ và công cụ sau để hỗ trợ thiết kế, phát triển, cài đặt và triển khai hệ thống một cách hiệu quả:

Visual Studio Code là trình soạn thảo mã nguồn phổ biến, hỗ trợ nhiều ngôn ngữ lập trình và tính năng như tự động hoàn thành, gỡ lỗi, tích hợp Git.

Python là ngôn ngữ lập trình bậc cao, dễ sử dụng và có hệ sinh thái thư viện phong phú, phù hợp cho việc phát triển các thuật toán tối ưu như bài toán Vehicle Routing Problem (VRP). Python hỗ trợ lập trình nhanh, dễ kiểm thử và dễ mở rộng.

Các thư viện **numpy**, **random**, **time**, **matplotlib** là các thư viện quan trọng trong Python. Trong đó, **numpy** cung cấp các cấu trúc dữ liệu dạng mảng (array)

và các phép toán số học hiệu quả, *random* dùng để tạo các tham số ngẫu nhiên, *time* để tính toán thời gian chạy thuật toán, còn *matplotlib* dùng để biểu diễn dưới dạng biểu đồ.

3.5. Xây dựng và cài đặt hệ thống

3.5.1. Xây dựng và cài đặt bộ quản lý sự kiện

Dựa theo kiến thức từ những ngôn ngữ lập trình đã học và mã giả đã xây dựng và phân tích ở phần Chương 2, bây giờ chúng em sẽ nghiên cứu và triển khai bộ quản lý sự kiện trong hệ thống dưới dạng tổng quát như sau:

Time := 0

$T_{CO} := 0.5 * T$

Khởi tạo PendOrds từ các đơn hàng ban đầu.

Committed := $\emptyset$

Khởi tạo pheromone.

While (PendOrds $\neq \emptyset$ hoặc Time < T_{CO})

Nếu Time > 0:

start_time := Time + T_{ac}

Ngược lại:

start_time := Time

StaticOrders := các đơn trong PendOrds có time $\leq$ start_time

Nếu StaticOrders rỗng:

Time := Time + T_{ts}

continue

Xây dựng demand từ StaticOrders

$(\text{BestSol}, \text{BestCost}) := \text{ACS}(\text{StaticOrders})$

$\text{CommOrds} := \emptyset$

Nếu BestSol tồn tại:

Với mỗi đơn o trong StaticOrders :

Nếu khách hàng của o thuộc BestSol :

thêm o vào CommOrds

$\text{PendOrds} := \text{PendOrds} \setminus \text{CommOrds}$

$\text{Committed} := \text{Committed} \cup \text{CommOrds}$

$\text{PendOrds} := \text{PendOrds} \cup \{\text{các đơn xuất hiện trong khoảng thời gian } T_{ts}\}$

$\text{Time} := \text{Time} + T_{ts}$

Cập nhật vị trí xe và sức chứa.

Cập nhật pheromone.

EndWhile

Đảm bảo tất cả xe quay về depot.

Trả về Committed.

3.5.2. Xây dựng và cài đặt thuật toán ACS

Dựa theo kiến thức từ những ngôn ngữ lập trình đã học và mã giả đã xây dựng và phân tích ở phần Chương 2, bây giờ chúng em sẽ nghiên cứu và triển khai thuật toán ACS trong hệ thống dưới dạng như sau:

Khởi tạo $\text{BestCost} := \infty$

$\text{BestSol} := \emptyset$

Nếu pheromone chưa tồn tại:

Khởi tạo $\tau_{ij} := \tau_0$ với mọi cạnh (i, j)

While (chưa đạt số vòng lặp tối đa)

For k := 1 đến m (số lượng kiến)

Khởi tạo kiến k tại depot.

While (kiến chưa hoàn thành lời giải)

Chọn khách hàng tiếp theo j từ tập khả thi.

Nếu không còn khách phù hợp:

thoát vòng lặp.

Di chuyển đến j.

Cập nhật pheromone cục bộ trên cạnh (i, j)

EndWhile

Đưa kiến quay về depot.

Tính chi phí lời giải hiện tại.

Nếu chi phí tốt hơn BestCost :

$\text{BestCost} :=$ chi phí hiện tại

$\text{BestSol} :=$ lời giải hiện tại

EndFor

Với mỗi cạnh (i, j) thuộc BestSol:

Cập nhật pheromone toàn cục.

EndWhile

Trả về BestSol, BestCost.

3.5.3. Xây dựng và cài đặt thủ tục bảo toàn pheromone

Dựa theo kiến thức từ những ngôn ngữ lập trình đã học và mã giả đã xây dựng và phân tích ở phần Chương 2, bây giờ chúng em sẽ nghiên cứu và triển khai thủ tục bảo toàn pheromone trong hệ thống dưới dạng như sau:

Khởi tạo ma trận pheromone mới τ_{new} với giá trị τ_0 .

For mỗi cạnh (i, j):

Nếu (i, j) tồn tại trong ma trận pheromone cũ:

$$\tau_{new} [i][j] := (1 - \gamma_r) * \tau_{old} [i][j] + \gamma_r * \tau_0$$

Ngược lại:

$$\tau_{new} [i][j] := \tau_0$$

EndFor

Trả về τ_{new} .

3.6. Kiểm thử và đánh giá hệ thống

3.6.1. Kiểm thử và đánh giá hệ thống với số lượng nhỏ khách hàng

Để đánh giá quá trình hoạt động của hệ thống, nhóm chúng em tiến hành kiểm thử với một bộ dữ liệu nhỏ gồm 3 khách hàng, nhằm quan sát trực quan quá trình xử lý và khả năng hội tụ của thuật toán như sau, được thiết lập với các tham số như sau:

Số lượng khách hàng: 3

Tọa độ các điểm (bao gồm depot và khách hàng) được thiết lập thủ công trên mặt phẳng 2 chiều.

Ma trận khoảng cách được xây dựng tương ứng giữa các điểm.

Các đơn hàng có **thời điểm xuất hiện** khác nhau:

Khách hàng 1: thời điểm 0.

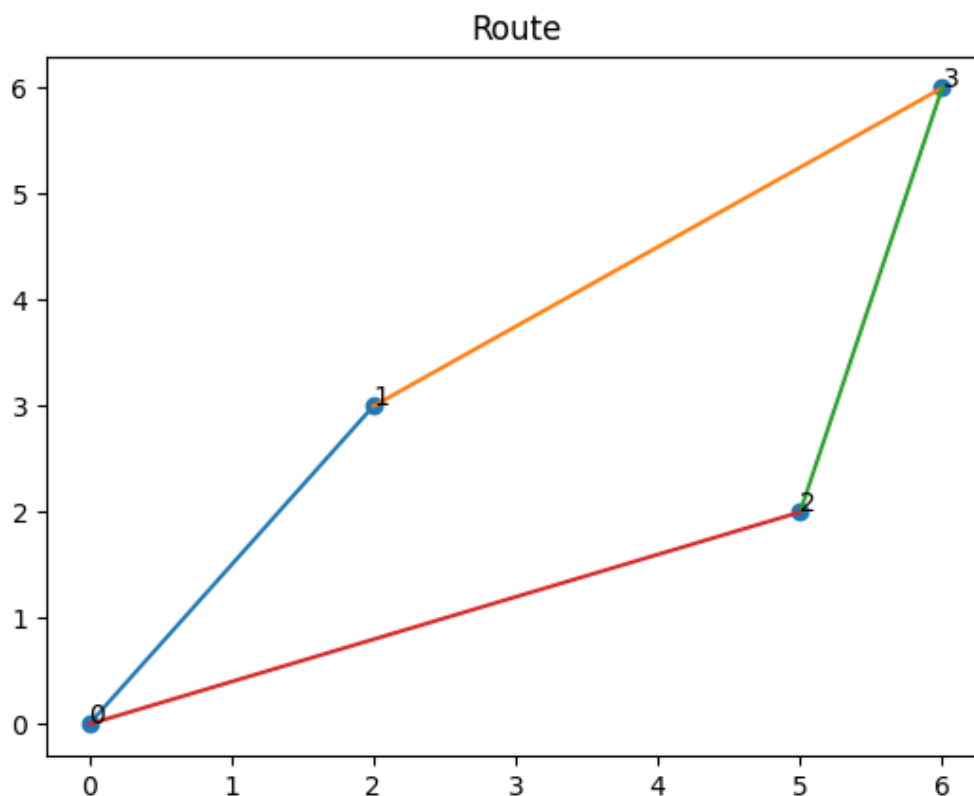
Khách hàng 2: thời điểm 5.

Khách hàng 3: thời điểm 10.

Nhu cầu mỗi khách hàng: 10 đơn vị.

Sức chứa phương tiện: 100 (đảm bảo phục vụ tất cả khách hàng trong một tuyến)

Hệ thống đã được xây dựng dựa trên mã giả của các thành phần trong thuật toán bao gồm bộ xử lý sự kiện, thuật toán ACS và cơ chế bảo toàn pheromone. Sau khi chạy chương trình, kết quả bài toán được biểu diễn dưới dạng hình ảnh bên dưới, với thời gian phục vụ là 750, 0 đơn hàng đang chờ và 3 đơn hàng đã được phục vụ. Kết quả kiểm thử với tập dữ liệu nhỏ cho thấy hệ thống hoạt động chính xác, ổn định và đáp ứng tốt yêu cầu của bài toán định tuyến xe động, là cơ sở để tiếp tục mở rộng và đánh giá hệ thống với các bộ dữ liệu có quy mô lớn hơn:



Hình 3.4. Biểu đồ biểu diễn kết quả kiểm thử với số lượng nhỏ khách hàng

3.6.2. Kiểm thử và đánh giá hệ thống với thuật toán GRASP

Để có thể đánh giá tổng quan hiệu năng giữa thuật toán GRASP và thuật toán ACS trong bài toán định tuyến xe động dựa trên cái nhìn của chúng em, dựa theo kết quả nghiên cứu của các tác giả, nhóm đã thực hiện xây dựng hệ thống so sánh thuật toán GRASP và thuật toán ACS với nền tảng đã thiết lập hệ thống thuật toán ACS đã có và tạo mới thuật toán GRASP với nền tảng cơ chế hoạt động đã nắm được ở phần trên.

Thuật toán GRASP được cài đặt theo cơ chế xây dựng lời giải tham lam có ngẫu nhiên. Trong mỗi bước, thuật toán lựa chọn khách hàng tiếp theo từ danh sách ứng viên tốt nhất nhằm tăng tính đa dạng của lời giải. Sau nhiều lần lặp, thuật toán chọn ra tuyến đường có chi phí nhỏ nhất. Để có thể đưa ra kết quả so sánh rõ ràng nhất, chúng em đã tăng số lượng trên bộ dữ liệu với: số lượng kiến lên 10,

số lượng khách hàng lên 6 với các thời điểm phát sinh yêu cầu khác nhau và ma trận khoảng cách giữa các điểm giao hàng. Tương tự như bảng kết quả nghiên cứu, nhóm em cũng tiến hành chạy so sánh giữa hai thuật toán 5 lần để có kết quả chuẩn xác nhất:

Problem	Min	Max	Avg GRASP-DVRP	Min	Max	Avg ACS-DVRP
Test-1	25.00	25.00	25.00	25.00	25.00	25.00
Test-2	25.00	29.00	26.60	25.00	25.00	25.00
Test-3	25.00	28.00	26.80	25.00	25.00	25.00
Test-4	28.00	29.00	28.80	25.00	25.00	25.00
Test-5	25.00	29.00	26.40	25.00	25.00	25.00

Hình 3.5. Biểu đồ biểu diễn kết quả kiểm thử với thuật toán GRASP

Qua 5 lần kiểm thử, thuật toán ACS luôn cho kết quả:

- Giá trị nhỏ nhất (Min) = 25.00.
- Giá trị lớn nhất (Max) = 25.00.
- Giá trị trung bình (Avg) = 25.00.

Điều này chứng tỏ thuật toán ACS có khả năng hội tụ rất tốt và duy trì được tính ổn định trong suốt các lần thực thi. Cơ chế cập nhật pheromone giúp thuật toán tăng cường khai thác các tuyến đường tối ưu đã tìm được trước đó, từ đó giảm sự dao động của lời giải.

Ngược lại, thuật toán GRASP-DVRP cho kết quả biến động hơn giữa các lần chạy:

- Giá trị Min dao động từ 25.00 đến 28.00.
- Giá trị Max dao động từ 25.00 đến 29.00.
- Giá trị Avg dao động từ 25.00 đến 28.80.

Điều này cho thấy GRASP chịu ảnh hưởng khá lớn bởi yếu tố ngẫu nhiên trong quá trình lựa chọn lời giải. Mặc dù trong một số lần chạy thuật toán vẫn tìm được lời giải tối ưu bằng ACS, tuy nhiên tính ổn định chưa cao và chất lượng lời giải trung bình vẫn thấp hơn.

Ngoài ra, hệ thống cũng cho thấy khả năng xử lý hiệu quả các đơn hàng động phát sinh theo thời gian thực. Tất cả khách hàng đều được phục vụ thành công và không tồn tại đơn hàng chờ sau khi kết thúc quá trình mô phỏng.

Kết quả thực nghiệm cho thấy thuật toán ACS-DVRP đạt hiệu năng ổn định và chất lượng lời giải tốt hơn so với thuật toán GRASP-DVRP trong bài toán định tuyến xe động. Việc kết hợp cơ chế pheromone trong ACS giúp cải thiện đáng kể khả năng tìm kiếm tuyến đường tối ưu so với phương pháp heuristic thông thường. Từ đó, ta chứng minh được tính khả thi của hệ thống và hiệu quả của thuật toán ACS trong việc giải quyết bài toán định tuyến xe động.

KẾT LUẬN

1. Kết quả đạt được

- Xây dựng thành công mô hình DVRP động kết hợp thuật toán ACS (Ant Colony System).
- Hệ thống xử lý được đơn hàng theo thời gian (dynamic orders) và cập nhật lộ trình.
- Đảm bảo ràng buộc tải trọng xe (vehicle capacity) khi xây dựng tour.
- Sinh được lộ trình hợp lệ và trực quan hóa bằng biểu đồ.

2. Đánh giá, nhận xét

- Thuật toán chạy ổn định, không bị vòng lặp vô hạn nhờ giới hạn *max_steps*.
- Cách tổ chức EventsManager hợp lý, mô phỏng gần với bài toán thực tế.
- Có áp dụng đầy đủ local update & global update pheromone của ACS.
- Thời gian chạy nhanh do giảm số kiến và số vòng lặp.

3. Hạn chế tồn đọng

- Chất lượng lời giải chưa tối ưu cao do số lượng kiến (*num_ants*) và số vòng lặp (*max_iter*) nhỏ.
- Chiến lược chọn đỉnh tiếp theo thuần greedy, chưa dùng xác suất, dễ mắc kẹt local optimum.
- Chưa xử lý nhiều xe (multi-vehicle), mới dùng 1 xe đơn giản.
- Cơ chế commit đơn hàng còn đơn giản, chưa phản ánh đầy đủ thực tế DVRP phức tạp.

TÀI LIỆU THAM KHẢO

- [1] **R. Montemanni, L.M. Gambardella, A.E. Rizzoli, A.V. Donati**, “*Ant Colony System for a Dynamic Vehicle Routing Problem*”, 2005.
- [2] **M. Gendreau and J.-Y. Potvin**, “*Dynamic vehicle routing and dispatching*”, Transportation Science, 1998.
- [3] **L. M. Hvattum, A. Løkketangen, and G. Laporte**, “*Dynamic vehicle routing with stochastic demands*”, 2004.
- [4] **M. Savelsbergh and M. Sol**, “*DRIVE: Dynamic routing and interactive vehicle dispatching*”, 1998.
- [5] **P. Kilby, P. Prosser, and P. Shaw**, “*Dynamic VRP: A study of heuristics*”, 1998.
- [6] **M. Guntsch and M. Middendorf**, “*Ant colony optimization for dynamic TSP*”, 2002.
- [7] **É. D. Taillard**, “*Parallel iterative search methods for vehicle routing problems*”, Networks, vol. 23, no. 8, pp. 661–673, 1994.
- [8] **N. Christofides and J. E. Beasley**, “*The period routing problem*”, Networks, vol. 14, no. 2, pp. 237–256, 1984.
- [9] **M. L. Fisher, R. Jaikumar, and L. N. Van Wassenhove**, “*A generalized assignment heuristic for vehicle routing*”, Networks, vol. 11, no. 2, pp. 109–124, 1981.
- [10] **L. M. Gambardella, É. D. Taillard, and G. Agazzi**, “*MACS-VRPTW: A multiple ant colony system for vehicle routing problems with time windows*”,
in New Ideas in Optimization, McGraw-Hill, 1999, pp. 63–76.
- [11] **M. G. C. Resende and C. C. Ribeiro**, “*Greedy randomized adaptive search procedures*”,

in Handbook of Metaheuristics, F. Glover and G. A. Kochenberger, Eds.
Boston, MA, USA: Springer, 2003, pp. 219–249.